

Protocolo de Transporte Confiável sobre UDP:

Análise Comparativa de Stop-and-Wait, Go-Back-N e Selective Repeat

Gabriel Rebello, Gustavo Souza e Igor Wolwacz

29 de junho de 2026

1 Introdução

Este trabalho apresenta a implementação e análise do protocolo SRTP (*Simple Reliable Transport Protocol*), um protocolo de transporte confiável construído sobre UDP. Foram implementadas três variantes de controle de erros e fluxo: Stop-and-Wait (SAW), Go-Back-N (GBN) e Selective Repeat (SR), todas compartilhando o mesmo cabeçalho de 9 bytes com verificação CRC32.

O relatório está organizado em duas partes: a Parte 1 analisa o comportamento do Stop-and-Wait sob diferentes condições de latência e perda; a Parte 2 compara as três variantes, incluindo cenários de reordenação de pacotes que evidenciam as diferenças fundamentais entre GBN e SR.

2 Descrição da Implementação

A implementação foi desenvolvida em Python 3 utilizando exclusivamente a biblioteca padrão (`socket`, `struct`, `zlib`, `argparse`). O mesmo binário suporta os três modos via argumento `-mode [saw|gbn|sr]`.

O cabeçalho de 9 bytes é composto pelos campos: SYN (1 bit), FIN (1 bit), SEQ (14 bits), ACK flag (1 bit), NACK (1 bit), ACK (14 bits), Length (8 bits) e CRC32 (32 bits). O campo `Length` codifica a semântica de fim de stream: 255 indica pacote intermediário; valores menores indicam o último pacote da transferência.

A sessão é estabelecida por um *handshake* de três vias (SYN / SYN+ACK / ACK), com negociação da janela efetiva como o mínimo entre os valores propostos pelos dois lados. O encerramento é realizado em dois passos (FIN / FIN+ACK). Pacotes com CRC32 inválido são descartados silenciosamente pelo receiver, sem envio de NACK; o timeout fixo de 100 ms no sender garante a retransmissão.

3 Parte 1 - Stop-and-Wait

3.1 Throughput teórico

No mecanismo Stop-and-Wait, o sender transmite um pacote e aguarda o ACK antes de enviar o próximo. A eficiência do protocolo é dada por:

$$\eta = \frac{T_{tx}}{T_{tx} + RTT} \quad (1)$$

onde T_{tx} é o tempo de transmissão do pacote e RTT é o tempo de ida e volta. O tamanho máximo de um pacote de dados é $9 + 255 = 264$ bytes. Considerando a largura de banda da rede WiFi utilizada nos testes (≈ 100 Mbit/s):

$$T_{tx} = \frac{264 \times 8}{100 \times 10^6} \approx 0,021 \text{ ms} \quad (2)$$

O RTT medido no cenário baseline via Wireshark foi de aproximadamente 9 ms, valor dominado pelo processamento e pela pilha UDP, não pela propagação. Aplicando a fórmula:

$$\eta = \frac{0,021}{0,021 + 9} \approx 0,23\% \Rightarrow \text{Throughput}_{max} \approx 230 \text{ Kbit/s} \approx 28,75 \text{ KB/s} \quad (3)$$

O throughput medido no cenário L0/P0 foi de 22,69 KB/s, representando cerca de 79% do valor teórico. A diferença é atribuída ao overhead do sistema operacional, da pilha de rede e do interpretador Python, que introduzem latência adicional não capturada pelo RTT médio do Wireshark.

3.2 Análise dos cenários de latência

Os cenários de latência foram testados com o software Clumsy configurado no receiver, adicionando atraso artificial nos pacotes UDP recebidos na porta 6000. Para cada cenário foi transferido um arquivo de 50.000 bytes (aproximadamente 196 pacotes de dados).

Tabela 1: Stop-and-Wait - cenários de latência

Cenário	Latência adicionada	Throughput medido	Retransmissões
L0	0 ms	22,69 KB/s	0
L1	50 ms	3,00 KB/s	241
L2	100 ms	1,91 KB/s	3781
L3	150 ms	1,38 KB/s	7758

A degradação de throughput é severa já em L1: com 50 ms de latência adicionada, o RTT total passa a ser de aproximadamente 59 ms, consumindo mais da

metade da janela de timeout de 100 ms. Qualquer variação de jitter empurra pacotes individuais além do limite, gerando as 241 retransmissões observadas.

A transição crítica ocorre entre L1 e L2: com 100 ms de atraso adicionado, o RTT total se aproxima ou supera o timeout de 100 ms em praticamente todos os pacotes. O resultado é a explosão de retransmissões de 241 para 3781 - um aumento de $15\times$ - enquanto o throughput cai apenas pela metade. Isso ocorre porque cada retransmissão bem-sucedida ainda entrega o pacote, mas custa um timeout adicional de espera.

Em L3, o RTT excede sistematicamente o timeout, e cada pacote frequentemente requer múltiplas retransmissões antes de ser confirmado, explicando as 7758 retransmissões com throughput de apenas 1,38 KB/s.

3.3 Análise dos cenários de perda

Os cenários de perda foram configurados no Clumsy com descarte aleatório de pacotes UDP. O timeout fixo de 100 ms garante a retransmissão de pacotes perdidos.

Tabela 2: Stop-and-Wait - cenários de perda

Cenário	Taxa de perda	Throughput medido	Retransmissões
P0	0%	22,72 KB/s	0
P1	1%	19,89 KB/s	4
P2	5%	11,42 KB/s	20
P3	10%	9,35 KB/s	45
P4	25%	4,91 KB/s	73

A perda afeta o SAW de forma mais suave do que a latência. Comparando P4 (25% de perda, 4,91 KB/s, 73 retransmissões) com L1 (50 ms de latência, 3,00 KB/s, 241 retransmissões), fica evidente que o gargalo fundamental do SAW é o RTT, não a taxa de perda. No SAW há apenas um pacote em trânsito por vez, portanto cada perda custa exatamente um timeout - não há efeito cascata.

A eficiência sob perda segue aproximadamente $\eta_p = (1-p) \cdot \eta_0$: para $p = 25\%$, espera-se throughput de $0,75 \times 22,72 \approx 17$ KB/s no caso ideal. O valor medido de 4,91 KB/s é inferior porque cada retransmissão consome um timeout completo de 100 ms de espera, penalizando desproporcionalmente o throughput.

4 Parte 2 - Go-Back-N e Selective Repeat

4.1 Análise comparativa sob latência

A janela deslizante reduz drasticamente o impacto da latência ao manter múltiplos pacotes em trânsito simultaneamente. Enquanto o SAW desperdiça todo o tempo de

propagação aguardando um ACK por vez, GBN e SR continuam enviando durante esse período.

Tabela 3: Throughput comparativo (KB/s) - cenários de latência

Protocolo	L0	L1	L2	L3
SAW	22,69	3,00	1,91	1,38
GBN (janela 4)	40,09	11,82	7,58	5,71
GBN (janela 16)	111,79	40,26	29,40	20,23
SR (janela 4)	31,61	9,42	7,75	5,12
SR (janela 16)	151,71	42,41	29,72	22,11

Com janela 16 em L3, o GBN atinge 20,23 KB/s contra 1,38 KB/s do SAW - uma diferença de $15\times$, demonstrando que a janela deslizante neutraliza efetivamente o custo da latência.

O impacto do tamanho de janela é igualmente pronunciado: GBN janela 16 supera GBN janela 4 em cerca de $3,5\times$ em todos os cenários de latência. Isso reflete diretamente o número de pacotes que o protocolo consegue manter em trânsito durante o RTT.

4.2 Análise comparativa sob perda

Tabela 4: Retransmissões e throughput - cenários de perda (janela 16)

Protocolo	P1 (1%)	P2 (5%)	P3 (10%)	P4 (25%)
GBN - retransmissões	32	263	336	1298
GBN - throughput (KB/s)	251,37	83,93	55,03	14,28
SR - retransmissões	30	111	149	181
SR - throughput (KB/s)	200,44	75,32	44,95	21,25

Sob baixa perda (P1), GBN e SR têm comportamento similar. A divergência torna-se significativa a partir de P2: enquanto o SR retransmite apenas o pacote faltante, o GBN retransmite toda a janela a partir do pacote não confirmado. Em P4 (25% de perda), essa diferença resulta em 1298 retransmissões para o GBN contra apenas 181 para o SR, uma razão de $7\times$.

Nota-se que o GBN apresenta throughput superior ao SR em P1–P3, apesar de mais retransmissões. Isso ocorre porque os ACKs cumulativos do GBN avançam a janela mais rapidamente do que os ACKs individuais do SR. Entretanto, em P4, o SR supera o GBN (21,25 vs 14,28 KB/s): o custo das retransmissões em cascata do GBN finalmente sobrepõe a vantagem dos ACKs cumulativos, revelando um ponto de cruzamento entre 25% e 10% de perda.

4.3 Análise comparativa sob reordenação

Tabela 5: Comportamento sob reordenação (janela 16)

Protocolo	R0 (0%)	R1 (10%)	R2 (25%)
GBN - throughput (KB/s)	274,81	153,87	184,97
GBN - retransmissões	0	1779	2451
SR - throughput (KB/s)	279,15	222,47	255,13
SR - retransmissões	0	15	38

Este é o cenário que melhor evidencia a diferença fundamental entre os dois protocolos. O GBN descarta qualquer pacote fora de ordem e exige que o sender retransmita toda a janela, tratando reordenação da mesma forma que perda. O SR, por sua vez, bufferiza pacotes fora de ordem e os entrega em sequência após o preenchimento da lacuna.

Em R2 (25% de reordenação), o GBN acumula 2451 retransmissões contra apenas 38 do SR, uma diferença de $64\times$. O throughput do SR em R2 (255,13 KB/s) permanece próximo ao baseline (279,15 KB/s), evidenciando que a reordenação quase não afeta o SR. Já o GBN cai para 184,97 KB/s em R2, com a variação entre R1 e R2 explicada pela aleatoriedade do WiFi entre as execuções.

Redes WiFi naturalmente reordenam pacotes UDP com maior frequência do que redes cabeadas, tornando o SR especialmente adequado para esse ambiente.

5 Conclusão

Os resultados obtidos permitem estabelecer condições claras de adequação para cada protocolo. O Stop-and-Wait é viável apenas em redes com RTT muito baixo e sem perda: seu throughput de 22,69 KB/s no baseline degrada para 1,38 KB/s com apenas 150 ms de latência adicional, tornando-o inadequado para qualquer rede com latência perceptível.

O Go-Back-N apresenta excelente throughput em redes com baixa perda e baixa reordenação, graças aos ACKs cumulativos que avançam a janela eficientemente. Porém degrada de forma agressiva sob perda ou reordenação elevadas: em P4, acumula 1298 retransmissões, e em R2, 2451, ambos reflexo do mecanismo de retransmissão em cascata que penaliza toda a janela por um único pacote fora de ordem.

O Selective Repeat demonstrou ser o protocolo mais robusto nas condições testadas. Embora apresente throughput ligeiramente inferior ao GBN em cenários sem perturbação, sua superioridade é inequívoca sob perda elevada (21,25 vs 14,28 KB/s em P4) e sob reordenação (255,13 vs 184,97 KB/s em R2, com 38 vs 2451 retransmissões). O custo adicional de um buffer no receiver é amplamente justificado pelo ganho de eficiência em redes reais, onde reordenação e perda são inevitáveis.